

Microcontrollers, Sensors and Communications

MICROCONTROLLERS, SENSORS AND COMMUNICATIONS

HALIMA ABIDEMI ATANDA

Contents

Design Overview	1
System Components	3
System Operation	3
Schematic Diagram	5
Software Design	5
Flowchart	6
System Security Considerations	7
Conclusion	8

Design Overview

The design overview of the system encompasses both hardware and software components, which provides a comprehensive understanding of how the system functions and interacts with its environment. The Bluetooth and CAN bus control system is designed to enable communication between an Arduino Uno microcontroller and external devices via Bluetooth and CAN bus. The system allows users to control the state of LEDs and receive button state information through Bluetooth communication. Additionally, the system communicates with external devices over the CAN bus protocol to exchange data. By considering the hardware and software design aspects outlined below, the system achieves its intended functionality and provides a versatile platform for local and remote interactions. The system follows a modular design approach, allowing for easy scalability, maintainability, and integration of additional features or peripherals.

Hardware Design Overview:

The hardware design consists of several components interconnected to form a cohesive system. These components include: an Arduino Uno microcontroller board, push buttons, LEDs, a potentiometer (for analog input), a CAN bus transceiver, and Bluetooth modules.

1. Arduino Uno Board:

This serves as the central processing unit of the system. It controls the interaction between various peripherals and interfaces. It also executes the firmware logic to manage inputs, outputs, and communication protocols.

2. Push Buttons:

This provides local input to the system. It is connected to the digital input pins of the Arduino Uno. It triggers specific actions when pressed, such as toggling LEDs or sending messages over the Bluetooth devices.

3. LEDs:

This serves as visual indicators of the system's state. It is connected to the digital output pins of the Arduino Uno. It is Controlled to illuminate or extinguish based on the input received or the received messages.

4. MCP2515 CAN Bus Module:

This facilitates communication with the CAN bus network. It is connected to the Arduino Uno via SPI communication protocol. It enables reception and transmission of CAN messages between the system and other nodes on the CAN bus.

5. Bluetooth Modules:

This enables wireless communication between the system and external devices. It utilizes the SoftwareSerial library to establish serial communication with the Arduino Uno. It also facilitates the bidirectional data exchange with a Bluetooth-enabled monitoring device or control interfaces, providing flexibility and mobility.

Software Design Overview:

The software design encompasses the firmware running on the Arduino Uno in C/C++ programming language using the Arduino IDE, it is responsible for controlling hardware peripherals, managing communication protocols, and executing application logic. Modularization facilitates code organization, reusability, and maintainability, which enables focus on individual components without affecting the overall system behavior. The firmware utilizes standard Arduino libraries, such as SPI for communication with peripherals, SoftwareSerial for Bluetooth communication, and custom libraries for CAN bus communication (e.g., MCP2515 library).

The key aspects of the software design include:

1. Input Handling:

Control flow within the firmware follows a state-based or event-driven model, where the system responds to external stimuli (e.g., button presses, incoming messages) by transitioning between different states or executing predefined actions.

2. Output Control:

This controls the state of the LEDs based on the users input, received messages, or the system conditions. It illuminates or extinguishes LEDs to provide visual feedback to the user.

3. CAN Bus Communication:

This implements the functionalities of sending and receiving CAN messages using the MCP2515 module. It listens for incoming CAN messages from the CAN bus network and processes them accordingly. It then generates and transmits CAN messages in response to the local events or user commands.

4. Bluetooth Communication:

This establishes serial communication with the Bluetooth modules to enable wireless data exchange. It listens for incoming messages from Bluetooth-enabled monitoring devices. It sends data over Bluetooth in response to local events or CAN bus messages.

5. Error Handling and Security:

This is Implemented by the error handling mechanisms to ensure robustness and reliability of the system. It validates received messages for integrity and authenticity to prevent unauthorized access or tampering.

Interaction with its Environment:

The system interacts with the software firmware through digital and analog input/output pins of the Arduino Uno in the following ways:

Local Interaction:

User inputs from push buttons and the potentiometer are read by the microcontroller and processed by the firmware to trigger specific actions or events. The firmware controls the state of LEDs by writing digital output values to corresponding pins, providing visual feedback to the user.

Remote Interaction (CAN Bus):

The system communicates with other nodes on the CAN bus network by sending and receiving CAN messages using appropriate communication protocols and libraries.

Remote Interaction (Bluetooth):

This enables wireless communication with external monitoring devices via Bluetooth and facilitates the data exchange and remote control using appropriate communication protocols and libraries.

System Components

The system components enable the Arduino-based system to provide local and remote functionality, user interaction, communication over the CAN bus and Bluetooth devices, and the visual feedback through LEDs.

Arduino Uno: This serves as the main microcontroller unit which is responsible for processing Bluetooth and CAN bus messages, as well as controlling LEDs and reading the button states. It provides the processing power and I/O capabilities necessary for interfacing with the various hardware components.

HC-05 Bluetooth Module: Two Bluetooth modules are used for wireless communication with external devices. They enable the Arduino Uno to establish Bluetooth connections with monitoring devices or other Bluetooth-enabled systems. The Bluetooth modules are connected to the Arduino Uno via digital pins for serial communication (UART).

MCP2515 CAN Bus Module: The MCP2515 module facilitates communication over the Controller Area Network (CAN) bus protocol. It allows the Arduino Uno to send and receive messages over the CAN bus, enabling communication with other CAN-enabled devices or systems. The module is connected to the Arduino Uno via SPI (Serial Peripheral Interface) for data exchange.

LEDs: Three Light-emitting diodes are connected to the Arduino Uno for visual indication and feedback. Each LED represents a specific status or condition within the system which are used to indicate the state of the system, such as whether it is powered on, connected to Bluetooth or CAN bus, or responding to user input.

Buttons: Three push buttons are connected to the Arduino Uno to provide local user input. Each push button allows users to trigger specific actions or events within the system. They are typically used for functions such as toggling LEDs, sending messages over Bluetooth or CAN bus, or triggering other events.

Potentiometer: This is connected to an analog pin, allowing for variable input values to be read and processed by the microcontroller.

Resistors: These are current-limiting resistors that are connected in series with the LEDs, buttons and Bluetooth.

Jumper Wires: These are cable wires that connects all the devices with Arduino uno

Breadboard: This device supports the use of jumper wires with no soldering needed.

Arduino IDE: This is the software environment in which all the coding that is needed is been written and it is configured to the Arduino Uno which all library used are installed on.

System Operation

This section outlines how the Arduino-based system functions both locally and remotely, utilizing the push buttons, LEDs, CAN bus communication, and Bluetooth connectivity. By effectively integrating local and remote functionality, the system provides users with a versatile and user-friendly interface for interacting with external devices or systems. The seamless operation of the system ensures reliability, responsiveness, and security in its communication and interaction with the surrounding environment.

Bluetooth Communication:

The system establishes Bluetooth connections with monitoring devices or and Bluetooth-enabled systems for wireless communication. It sends and receives messages over Bluetooth which triggers actions within the system, such as controlling LEDs, sending CAN messages, enabling bidirectional communication with external devices or updating system parameters. The system ensures secure communication by implementing encryption, authentication, and data validation mechanisms.

CAN Bus Communication:

The system communicates with external devices or systems via the Controller Area Network (CAN) bus protocol. It listens for incoming CAN messages on the CAN bus, processes and responds accordingly relevant messages. Messages received over the CAN bus may include requests for information, commands to perform specific actions, or status updates from other nodes on the network. The system filters incoming messages to ignore irrelevant or unauthorized messages which ensures data integrity and security.

Local Functionality:

The system operates locally by accepting input from three push buttons and providing visual feedback through three LEDs.

Accept Input from Push Buttons:

Implemented functionality reads input from three push buttons connected to the Arduino. These are configured as inputs with internal pull-up resistors enabled to ensure stable readings. By pressing a button, it toggles the state of an LED which changes the system mode and initiate a local operation.

Control LEDs:

Implemented functionality controls three LEDs connected to the Arduino. LEDs are configured as outputs, and their states are updated based on button inputs or received commands.

Accept Input from Analog Input:

Implemented functionality reads input from an analog sensor connected to one of the Arduino's analog input pins. The analog input (potentiometer) is used to sense environmental variables which provides additional local input to the system, such as controlling the brightness of the LED.

Remote Functionality (CAN):

Connect to CAN Bus:

The integrated MCP2515 CAN bus module connects the Arduino to the CAN bus network. which utilizes the appropriate CAN bus libraries to establish communication and handle message transmission and reception.

Receive CAN Messages:

This is implemented to receive CAN messages from the assessment dashboard or other CAN bus devices. Which uses the MCP2515 library to read incoming CAN messages and process them accordingly.

Ignore Inappropriate Messages:

Logics are implemented to filter out and ignore CAN messages that are not relevant to the system's operation. This ensures that only messages addressed to the system or containing relevant data are processed.

LED Control via CAN:

Functions are implemented to switch LEDs on or off based on CAN messages received from the dashboard. It processes incoming CAN messages and adjust LED states accordingly.

Send CAN Messages in Response to Local Inputs:

Implemented logic generate and send appropriate CAN messages to the dashboard in response to local inputs (button presses or analog sensor readings). It uses sent CAN messages which contains relevant data such as button states or sensor readings to update the dashboard.

Remote Functionality (Bluetooth):

Connecting to Bluetooth Monitor:

The HC-05 Bluetooth module is used to establish a connection with the assessment Bluetooth monitor. Which implements the auto-reconnect functionality to ensure seamless communication even after disconnection.

Send Messages via Bluetooth:

Logics are used to send appropriate messages via Bluetooth when specific CAN messages are received or when local inputs change. It Sends Bluetooth messages containing relevant data to control LEDs or update remote systems.

Receive Bluetooth Messages:

Functions are used to receive Bluetooth messages from the assessment Bluetooth monitor which processes the incoming Bluetooth messages to adjust local LED states or forward relevant information to the CAN bus.

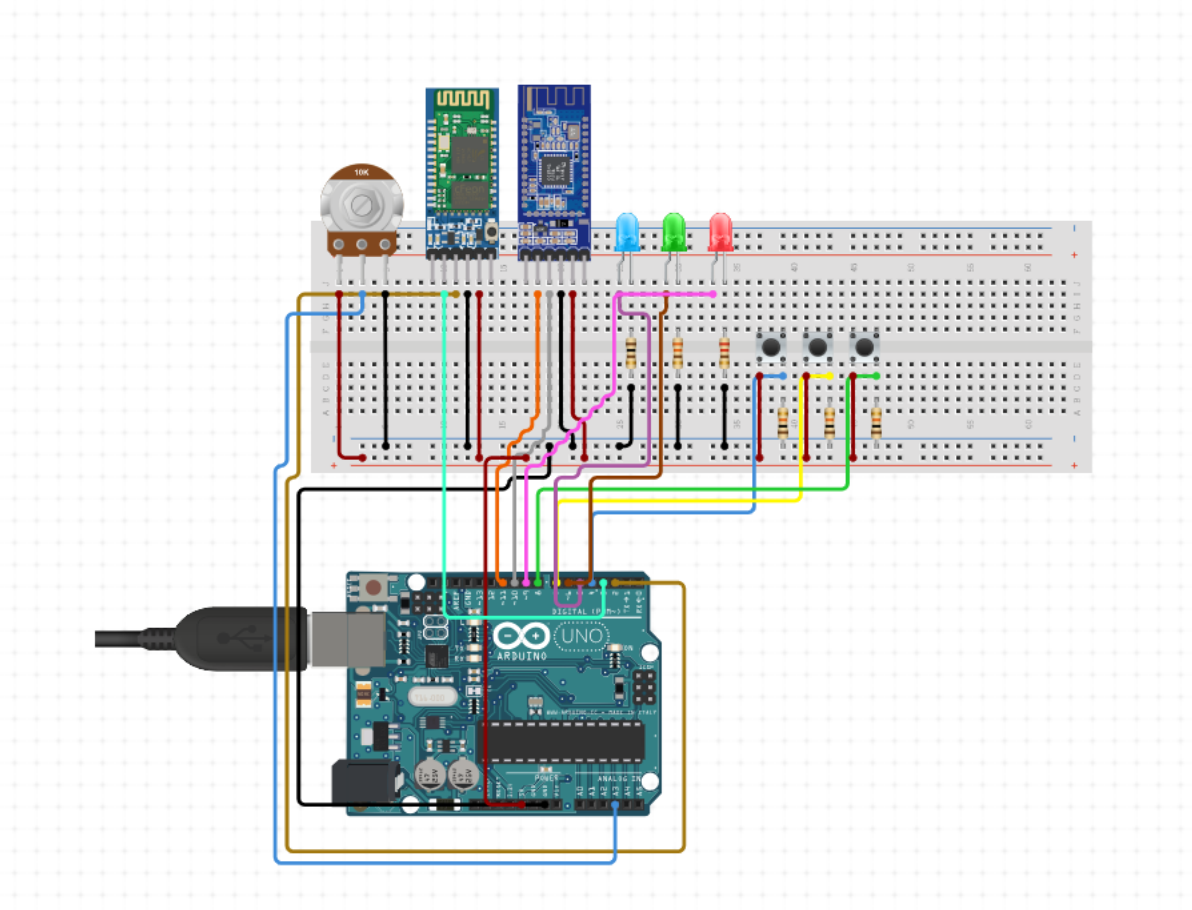
Integration of Local and Remote Functionality:

The system seamlessly integrates local and remote functionality to provide a comprehensive user experience. Local user input triggers actions within the system, such as changing LED states or sending messages over the CAN bus or Bluetooth. Incoming messages from external devices or systems via the CAN bus or Bluetooth also influences the system's behaviour, such as updating LED states or initiating local operations. The system maintains consistency, reliably and responsiveness across local and remote interactions ensuring that commands are executed accurately and efficiently without causing interference to other systems on the CAN bus network. Implemented error handling and validation mechanisms prevents data corruption and transmission errors.

Schematic Diagram

Arduino Uno schematic Diagram

Note: I could not find a Can Bus model for the illustration hence why it is not represented in the diagram.



Software Design

This section describes the structure and functionality of the Arduino code. The code is structured as follows:

Main Program Structure

The main program structure consists of the ``setup()`` and ``loop()`` functions.

Setup Function: Initializes tasks such as serial communication, Bluetooth module, CAN bus module, LED pins, button pins and initializing the libraries.

Loop Function: This contains the main execution loop where the program continuously listens for incoming Bluetooth and CAN bus messages. Processes incoming messages, updates LED states based on button presses, and sends response messages over Bluetooth or CAN bus.

Initialization

In the ``setup()`` function, the Arduino initializes serial communication for debugging purposes and sets up the pins for push buttons, LEDs, potentiometer, and CAN transceiver. Libraries such as SPI, MCP2515 (CAN bus), and SoftwareSerial (for Bluetooth communication) are initialized.

Message Processing

Incoming messages from both CAN bus and Bluetooth modules are processed in the ``loop()`` function. Messages are then checked for validity and parsed to extract message type and data, depending on the message type, appropriate actions are taken, such as controlling LEDs, sending CAN messages, or updating the state of local components (LEDs and buttons).

CAN Bus Communication

The Arduino micro controller communicates with other devices on the CAN bus using the MCP2515 library. Which listens for incoming CAN messages, processes them accordingly and responses to CAN messages are generated based on local inputs or Bluetooth commands which are sent back over the CAN bus.

Bluetooth Communication

Bluetooth communication is handled using the SoftwareSerial library to communicate with both master and slave Bluetooth modules. Incoming Bluetooth messages are processed similarly to CAN messages, extracting message type and data, and taking appropriate actions. Responses to Bluetooth messages also involves controlling local components or sending CAN messages.

Error Handling

Error handling mechanisms are implemented to handle situations such as invalid messages, communication failures, or unexpected behavior. Diagnostic messages are printed over the serial monitor interface for debugging purposes.

Modularity and Code Structure

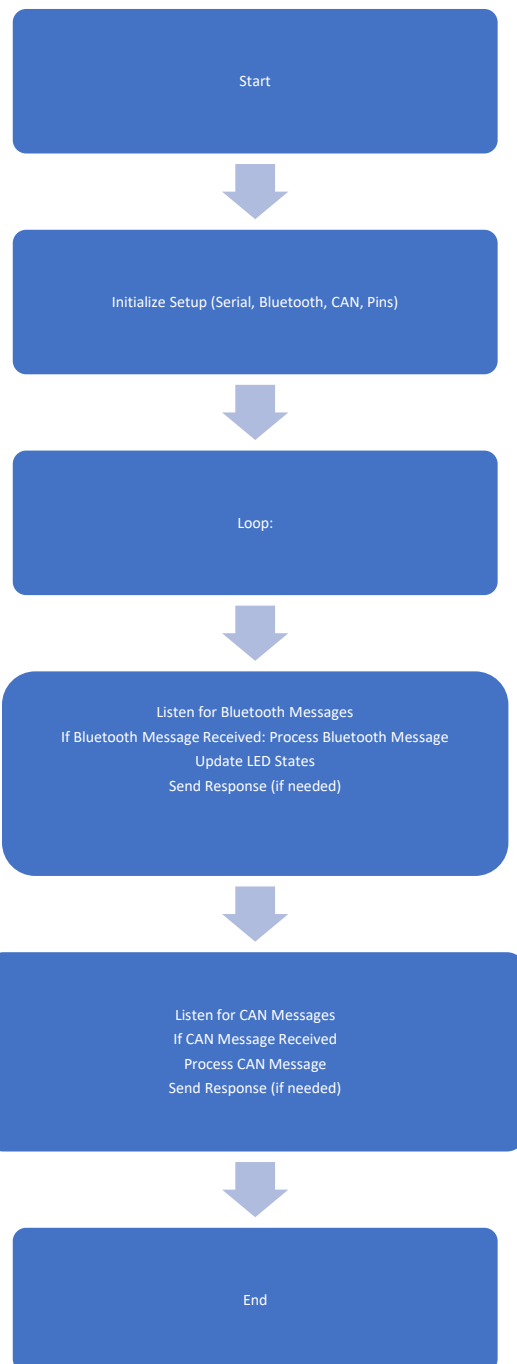
The code is structured into functions to encapsulate different tasks and promote modularity. Functions are used for tasks such as processing CAN messages, Bluetooth messages, checking button states and controlling LEDs.

Flow Control

Flow control mechanisms, such as conditional statements (if-else, switch-case) and loops, are used to control the flow of execution based on different conditions and inputs.

Flowchart

This section provides a graphical representation of the control flow and decision-making process within the Arduino code.



System Security Considerations

This section focuses on the measures taken to ensure the security and integrity of the system, particularly regarding communication over the CAN bus and Bluetooth interfaces. By addressing these security considerations, the system robustness and resilience against potential threats and attacks is assured, which ensures the safety and integrity of data transmitted over both the CAN bus and Bluetooth interfaces. Here are some aspects that were considered:

Data Integrity:

This ensures that data transmitted over both the CAN bus and Bluetooth interfaces are accurate and not tampered with. Implementation of checksums or CRC (Cyclic Redundancy Check) verifies the integrity of the received messages.

Authentication and Authorization:

Implementing authentication mechanisms ensures that only authorized devices can access the system. For Bluetooth communication, the use of pairing, binding and authentication protocols establishes secure connections between devices.

Encryption:

The encryption algorithms secure sensitive data transmitted over Bluetooth. Using protocols like Bluetooth Secure Simple Pairing (SSP) or Bluetooth Low Energy (BLE) security modes encrypts data and prevents eavesdropping or unauthorized access.

Secure Configuration:

This ensures that the default configurations of the CAN bus and Bluetooth interfaces are changed to prevent unauthorized access. Setting strong passwords and encryption keys where applicable and disabling unused features or services to reduce the attack surface.

Filtering:

Implementing filtering mechanisms to block unauthorized access or malicious traffic on both the CAN bus and Bluetooth interfaces. Using software filters to inspect incoming messages and enforce access control policies.

Firmware Updates:

Regularly updating firmware and software components to patch security vulnerabilities ensures that the system is protected against known exploits. Implementing secure firmware update mechanisms prevents unauthorized modification of device firmware.

Physical Security:

Protecting the physical integrity of the system secures access to hardware components and prevents unauthorized tampering. Using tamper-evident seals and enclosures detects any attempts at physical intrusion or tampering.

Logging and Auditing:

Implementation of logging and auditing mechanisms records and monitors system activities, including incoming messages, command executions, and access attempts. Maintaining audit trails facilitates forensic analysis in case of security incidents.

Secure Communication Protocols:

The use of secure communication protocols such as TLS (Transport Layer Security) or SSH (Secure Shell) for remote access to the system ensures that sensitive data transmitted over the network is encrypted and authenticated to prevent interception or manipulation.

Regular Security Assessments:

Conducting regular security assessments and penetration testing identifies and remediates vulnerabilities in the system. Performing vulnerability scans, code reviews, and risk assessments proactively address security weaknesses.

Conclusion

The implemented system successfully provides both local and remote functionality, allowing for control of LEDs and reception of input from push buttons and analog sensors. It seamlessly integrates with both the CAN bus network and Bluetooth module, enabling bidirectional communication and data exchange. The system's reliability and robustness ensure stable operation without interfering with other systems on the CAN bus network. The overall system meets the specified requirements and provides a solid foundation for further development and integration into various applications making it suitable for automotive, industrial, and home automation projects.